Introduction

Big Data Analytics

References & Grading

- **References**
  - “Data Mining – Concepts and Techniques” by Jiawei Han, Micheline Kamber, Jian Pei, third edition

References & Grading

- **References**
  - “Data Mining – Concepts and Techniques” by Jiawei Han, Micheline Kamber, Jian Pei, third edition
  - “Neural Networks and Deep Learning” (search neuralnetworksanddeeplearning on the Web)

References & Grading

- **References**
  - “Data Mining – Concepts and Techniques” by Jiawei Han, Micheline Kamber, Jian Pei, third edition
  - “Neural Networks and Deep Learning” (search neuralnetworksanddeeplearning on the Web)
- **Grading**
  - Mid-term exam 48%
  - Final exam: 50%
  - 2 Homeworks: 2%

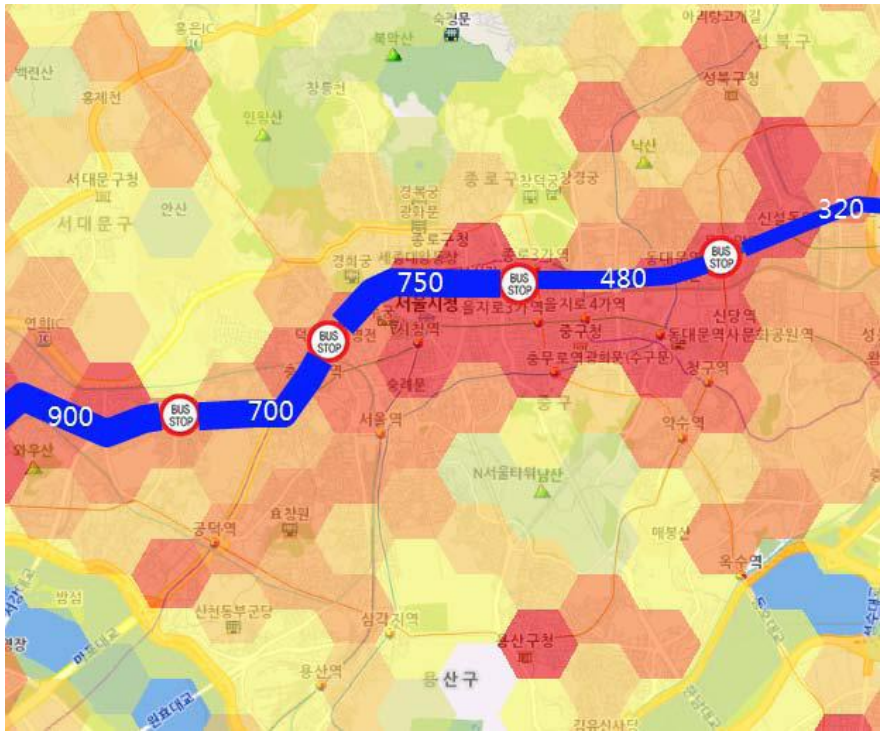
This Class

- Basic Concepts of Big Data Processing
  - Traditional way of data processing
  - Hadoop
    - Example: Word count, Page Rank, K-means clustering
  - Spark
- Basic Concepts of Analytics
  - Frequent Patterns
  - Classification
  - Neural Network
- Real world projects in telco business

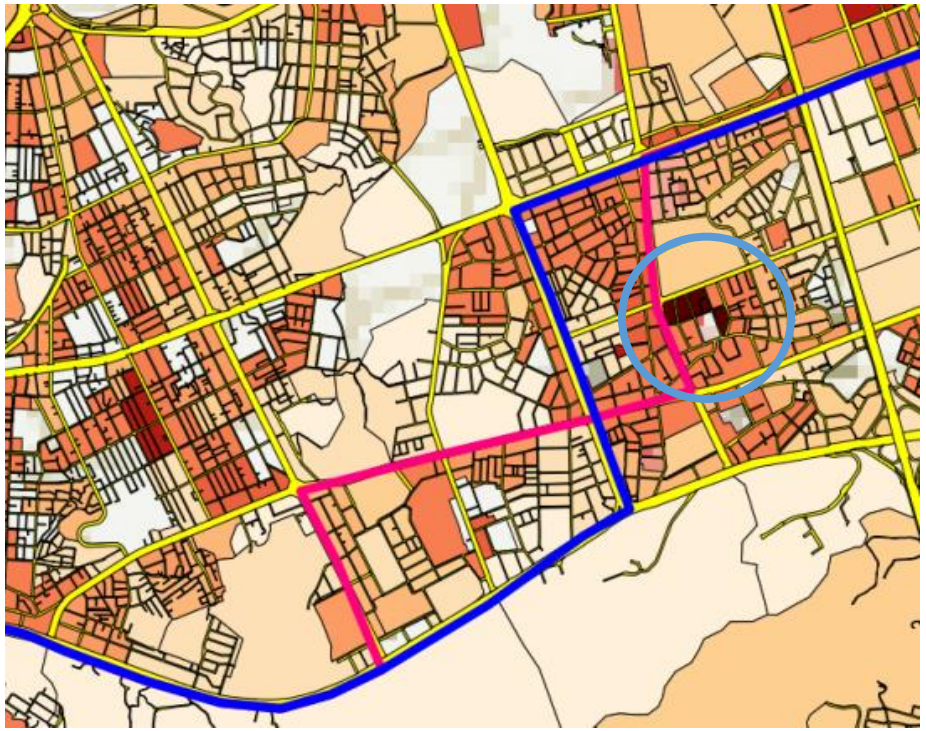
Let’s start with an example

Midnight bus routing in Seoul, Korea

of people in a region - # of people living in the reqion

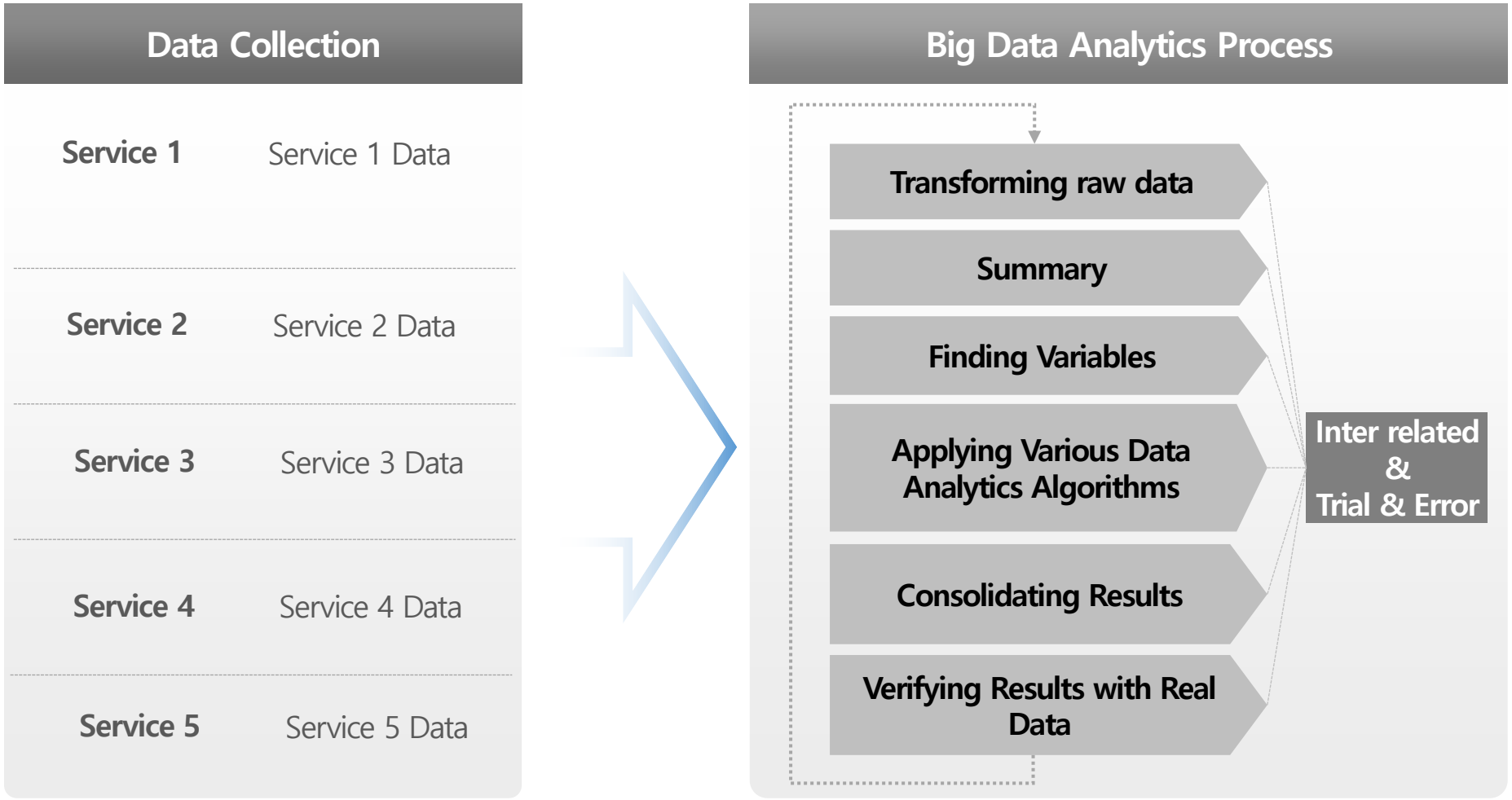


① Existing Route



② Improved Route

Big Data Analytics Process



{“S_id”: “xx”, “R_id”: “yy”, “S_time”: “20201010150021”, “E_time”: “20201010150313”, “S_cid”: “aa”, “R_cid”: “bb”}

Log 규격 문서에 탑재된 LOG 종류

```
- 레포트 요청 -
<?xml version="1.0" encoding="UTF-8"?>
<MAS method="req_report">
  <ServiceProviderID>test_sp</ServiceProviderID> -- SP ID
  <EndUserID>jjm</EndUserID> -- End-UserID
  <Result>0</Result> -- 메시지전달결과입니다. "0"성공 그 이외의 값(에러코드참조)
  <Time>20100126160953</Time> -- 메시지가 전달된 시간
  <FinishPage>0</FinishPage> -- FMS 전송의 경우, 전송된 마지막 페이지 수
  <Duration>0</Duration> -- VMS 전송의 경우 통화 시간(초)
  <CustomMessageID>11178</CustomMessageID> -- SP 가 발급한 ID
  <JobID>132858648</JobID> -- 서버에서 할당한 메시지에 대한 유일한 key 값.
  <GroupID></GroupID> -- MCS 가 생성한 동보/대량 전송 요청 구분 ID
  <SequenceNumber>1</SequenceNumber> -- 메시지전송 요청 내에서의 순서
  <MessageType>4</MessageType> -- 메시지유형입니다."4"MMS
  <SendNumber> x99qRyDwUPrFtphMADbvwx9s=</SendNumber> -- 전송번호 (복호화)
  <ReceiveNumber> x89qRyDwUPrFtphMADbvw=s=</ReceiveNumber> -- 수신번호 (복호화)
  <CallbackNumber> x96BqRyDwUPrFtphMVDbvw==</CallbackNumber> -- 회신번호(복호화)
  <RelpyInfo></RelpyInfo> -- 시나리오전송 결과
  <TelcoInfo>1</TelcoInfo> -- 이통사 번호. "1"SKT
  <Fee>55</Fee> -- 과금 기준요금
```

실제 AP 상에 기록되었던 Log 파일(2018-04-08)

```
[110057.453][][Success] [Inner] SendReport(Report[SP_ID:shcon20003, EU_ID:shcon20003,
JobID:66069210995, C_MsgID:20180407191434358018], unreach:true, send to EU:true)

[110057.453][][Success] [Inner] _SendUnreachReport(jobID:66069210995, result:0, sp:false,
option:1)

[110057.453][MRSC_/119.205.196.140:11073][Success] OnIoComplete : 807 bytes received
[110057.453][MRSC_/119.205.196.140:11073][Info] XML Received
<?xml version="1.0" encoding="UTF-8"?>
<RCP method="completed">
 <JobID>66072232579</JobID>
 <REQREPORT>
  <SequenceNumber>72232579</SequenceNumber>
  <Result>0</Result>
  <MsgID></MsgID>
  <Time>20180408110058</Time>
  <SP_ID></SP_ID>
  <EU_ID></EU_ID>
  <GRP_ID></GRP_ID>
  <C_MSG_ID>978721769</C_MSG_ID>
  <ReceiveNumber></ReceiveNumber>
  <SubmitTime>20180408110054</SubmitTime>
  <ExpireTime>20180408140054</ExpireTime>
  <SUB_GRP_ID>1</SUB_GRP_ID>
  <MSG_TYPE>1</MSG_TYPE>
  <TECOINFO>3</TECOINFO>
  <RETRY_COUNT>0</RETRY_COUNT>
  <SendNumber>030809043300</SendNumber>
  <CallbackNumber></CallbackNumber>
  <RD>01990000000</RD>
  <ONSE>0</ONSE>
  <StatusText></StatusText>
 </REQREPORT>
</RCP>
[110057.453][][Success] [Inner] SendReport(Report[SP_ID: EU_ID:
JobID:66072232579, C_MsgID:978721769], unreach:false, send to EU:false)

[110057.453][][Success] [Inner] _SendUnreachReport(jobID:66072232579, result:0, sp:true,
option:1)
```


```
{"result":"","jobid":"66072232606","host":"","logid":2,"time":"","c_msg_id":"758595208#!900222",
"type":"xml-RCP-consume-REQSMS"}
```

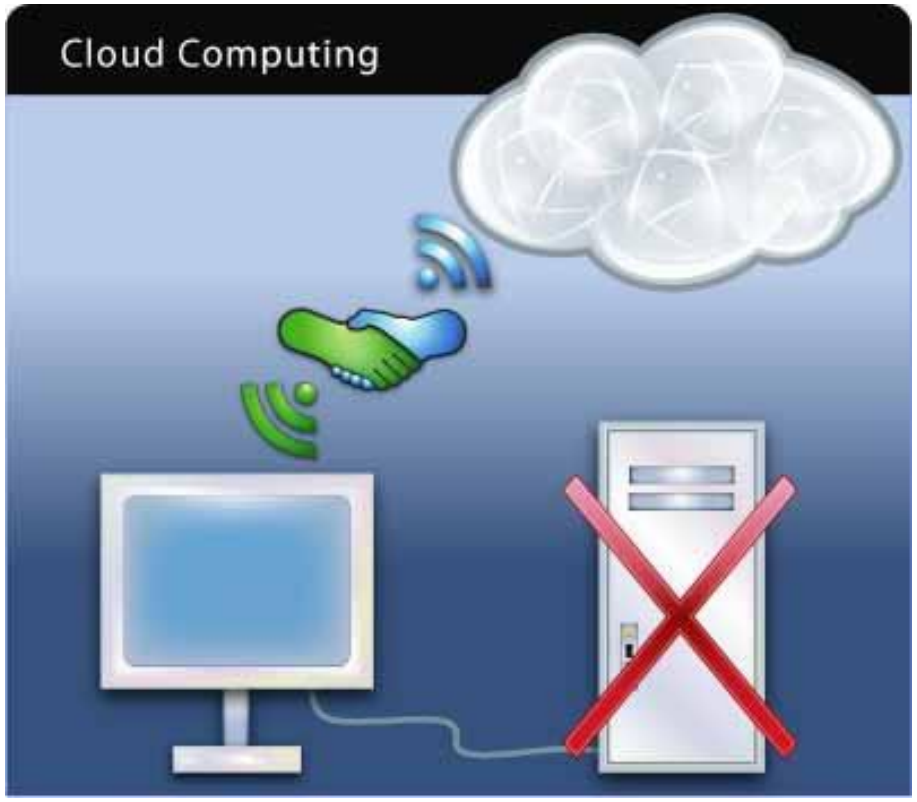
Traditional Data Processing





Cloud Computing

Complicated data processings which take massive computing power and data storages are executed on the central server (Cloud) rather than on personal PC's or local servers.





Cloud Computing - Google

◆ Google adopted service platforms based on cloud computing with distributed file systems, not commercial DBMS from the beginning of Google foundation, which have big advantages in terms of cost and scalability

◆But it had a disadvantage that it is very hard to develop parallel applications

Young Google's Concern

◆ It's a road to success if they can manipulate/analyze whole data in the Web

◆ The size of the Web data increases twice every year (Maybe more that twice now!!)

- They need Entire Web Scale data processing capabilities
- Hugh increase in HW/SW cost

◆ DBMS Restriction

- Only Scalable upto a certain point: OLTP, DW
- Doesn't fit Big Data Processings
- Very expensive to acquire good DB, and operation cost is also very expensive
- If the DB is not powerful enough to process all the data, a new powerful server should be puchrsed

◆ Google entered search engine business using Cloud Computing platforms to acquire competitive edges by spending much less HW/SW cost compared to Yahoo.

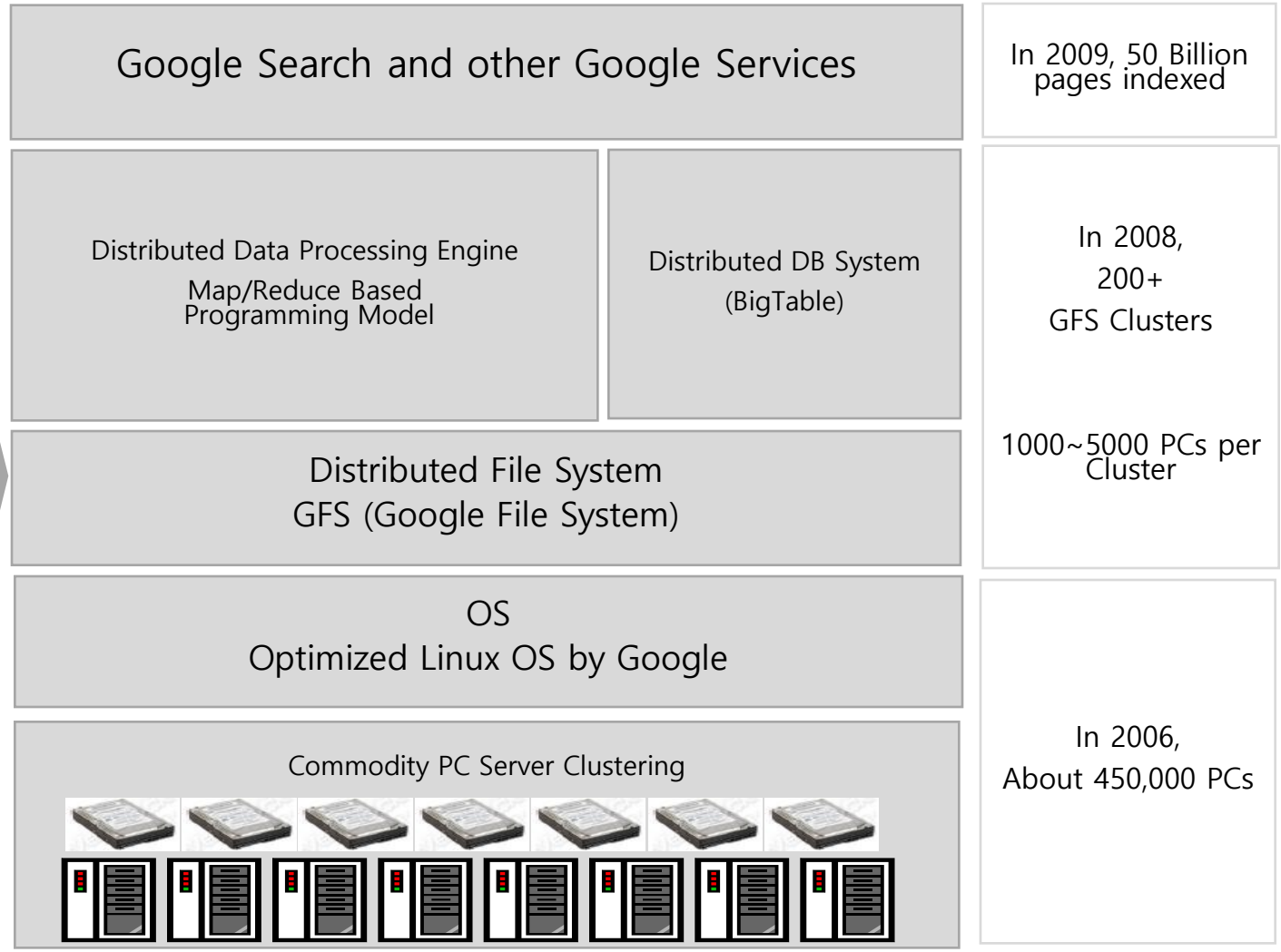
◆ The only problem is that they need to develop scalable, fault-tolerant, parallel software, which is very hard

Cloud Computing

Google Cloud Computing Platform

Parallel Computing Major Challenges

- **Transparency**
  It's very hard to build distributed programs. Need a consistent programming model which is easy to program to process huge data
- **Fault Tolerance**
  Lots of Commodity Devices cause lots of Troubles
- **Scalability**
  More data needs more HW in low cost



In 2009, 50 Billion pages indexed

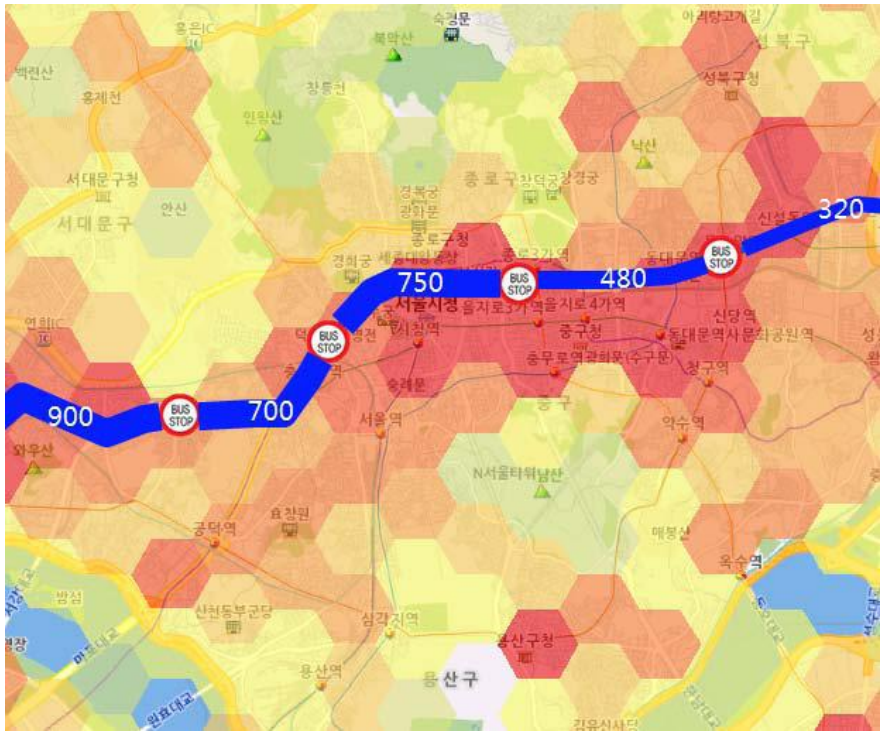
In 2008,
200+
GFS Clusters

1000~5000 PCs per Cluster

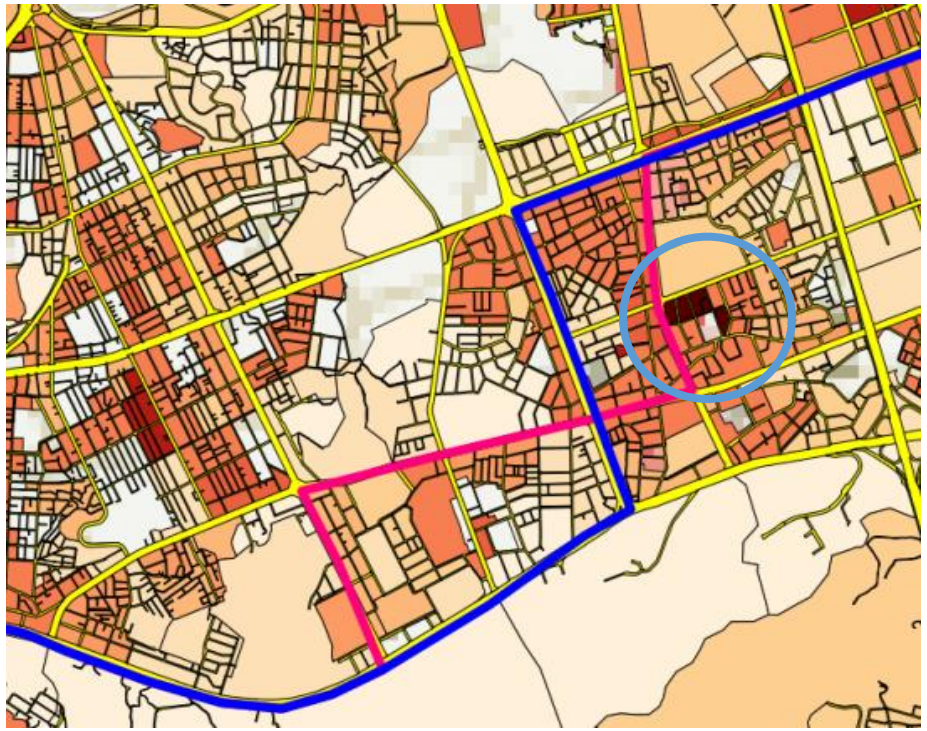
In 2006,
About 450,000 PCs

Midnight bus routing in Seoul, Korea

of people in a region - # of people living in the reqion



① Existing Route



② Improved Route

Log 규격 문서에 탑재된 LOG 종류

```
- 레포트 요청 -
<?xml version="1.0" encoding="UTF-8"?>
<MAS method="req_report">
  <ServiceProviderID>test_sp</ServiceProviderID> -- SP ID
  <EndUserID>jjm</EndUserID> -- End-UserID
  <Result>0</Result> -- 메시지전달결과입니다. "0"성공 그 이외의 값(에러코드참조)
  <Time>20100126160953</Time> -- 메시지가 전달된 시간
  <FinishPage>0</FinishPage> -- FMS 전송의 경우, 전송된 마지막 페이지 수
  <Duration>0</Duration> -- VMS 전송의 경우 통화 시간(초)
  <CustomMessageID>11178</CustomMessageID> -- SP 가 발급한 ID
  <JobID>132858648</JobID> -- 서버에서 할당한 메시지에 대한 유일한 key 값.
  <GroupID></GroupID> -- MCS 가 생성한 동보/대량 전송 요청 구분 ID
  <SequenceNumber>1</SequenceNumber> -- 메시지전송 요청 내에서의 순서
  <MessageType>4</MessageType> -- 메시지유형입니다."4"MMS
  <SendNumber> x99qRyDwUPrFtphMADbvwx9s=</SendNumber> -- 전송번호 (복호화)
  <ReceiveNumber> x89qRyDwUPrFtphMADbvw=s=</ReceiveNumber> -- 수신번호 (복호화)
  <CallbackNumber> x96BqRyDwUPrFtphMVDbvw==</CallbackNumber> -- 회신번호(복호화)
  <RelpyInfo></RelpyInfo> -- 시나리오전송 결과
  <TelcoInfo>1</TelcoInfo> -- 이통사 번호. "1"SKT
  <Fee>55</Fee> -- 과금 기준요금
```

실제 AP 상에 기록되었던 Log 파일(2018-04-08)

```
[110057.453][][Success] [Inner] SendReport(Report[SP_ID:shcon20003, EU_ID:shcon20003,
JobID:66069210995, C_MsgID:20180407191434358018], unreach:true, send to EU:true)

[110057.453][][Success] [Inner] _SendUnreachReport(jobID:66069210995, result:0, sp:false,
option:1)

[110057.453][MRSC_/119.205.196.140:11073][Success] OnIoComplete : 807 bytes received
[110057.453][MRSC_/119.205.196.140:11073][Info] XML Received
<?xml version="1.0" encoding="UTF-8"?>
<RCP method="completed">
 <JobID>66072232579</JobID>
 <REQREPORT>
  <SequenceNumber>72232579</SequenceNumber>
  <Result>0</Result>
  <MsgID></MsgID>
  <Time>20180408110058</Time>
  <SP_ID></SP_ID>
  <EU_ID></EU_ID>
  <GRP_ID></GRP_ID>
  <C_MSG_ID>978721769</C_MSG_ID>
  <ReceiveNumber></ReceiveNumber>
  <SubmitTime>20180408110054</SubmitTime>
  <ExpireTime>20180408140054</ExpireTime>
  <SUB_GRP_ID>1</SUB_GRP_ID>
  <MSG_TYPE>1</MSG_TYPE>
  <TECOINFO>3</TECOINFO>
  <RETRY_COUNT>0</RETRY_COUNT>
  <SendNumber>030809043300</SendNumber>
  <CallbackNumber></CallbackNumber>
  <RD>01990000000</RD>
  <ONSE>0</ONSE>
  <StatusText></StatusText>
 </REQREPORT>
</RCP>
[110057.453][][Success] [Inner] SendReport(Report[SP_ID: EU_ID:
JobID:66072232579, C_MsgID:978721769], unreach:false, send to EU:false)

[110057.453][][Success] [Inner] _SendUnreachReport(jobID:66072232579, result:0, sp:true,
option:1)
```


```
{"result":"","jobid":"66072232606","host":"","logid":2,"time":"","c_msg_id":"758595208#!900222",
"type":"xml-RCP-consume-REQSMS"}
```

{“S_id”: “xx”, “R_id”: “yy”, “S_time”: “20201010150021”, “E_time”: “20201010150313”, “S_cid”: “aa”, “R_cid”: “bb”}

Contents

1. Cloud Computing
2. Map-Reduce
    - Motivation
    - Key Idea of Map/Reduce
    - Word Count example
    - Sort example
    - Inverted Index example
    - Google Page Rank example
    - Matrix example
    - Impact of Map/Reduce on Google
3. Hadoop
4. Big Data Mining

Key Idea of Map/Reduce

- **Data type:** **key-value *records***
- **Map function:**

$$(K_{in}, V_{in}) \rightarrow \square\ list(K_{inter}, V_{inter})$$

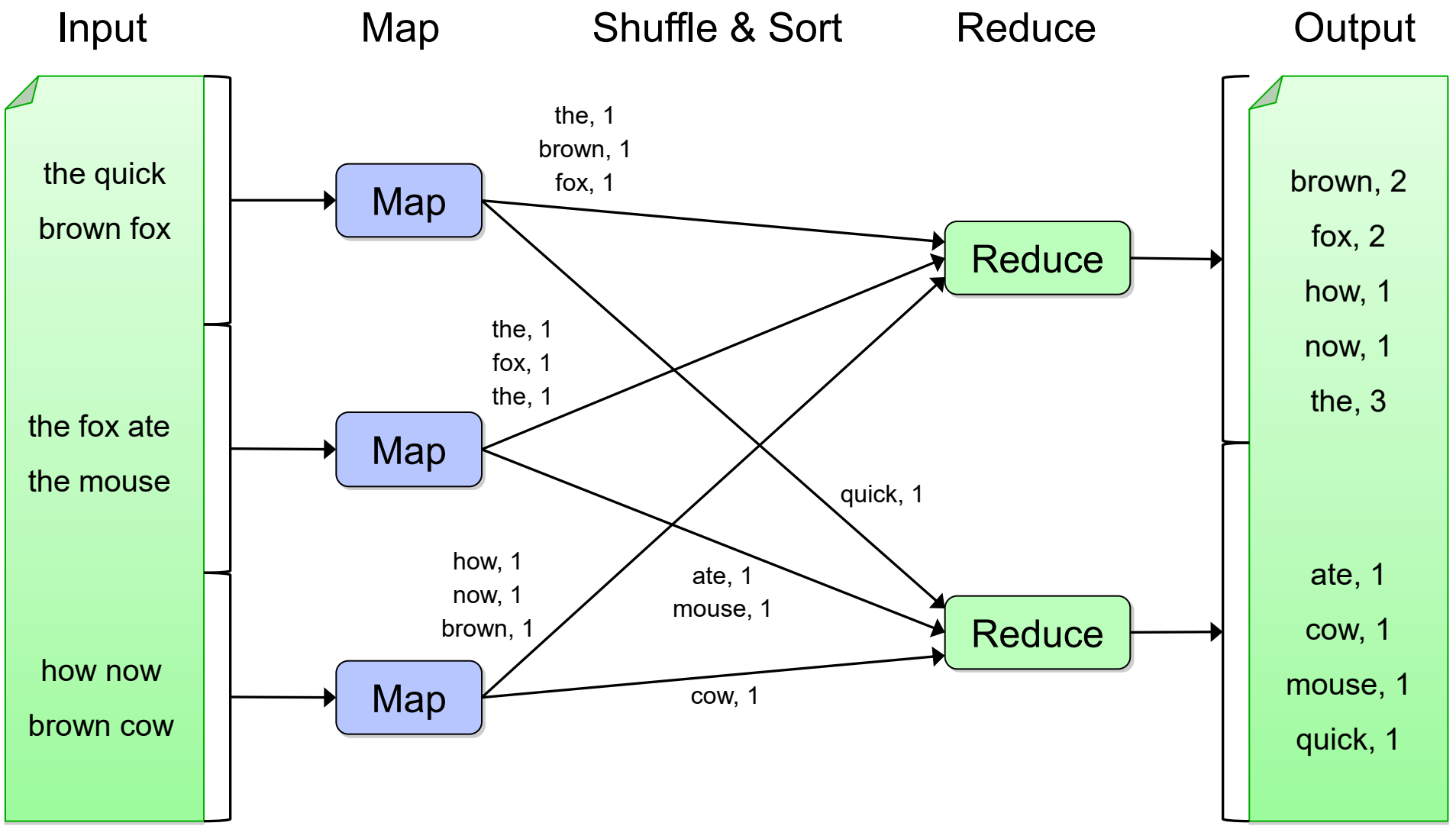
- **Reduce function:**

$$(K_{inter}, list(V_{inter})) \rightarrow \square\ list(K_{out}, V_{out})$$

◆ Hadoop Platform supports communications between nodes
- Failure recovery
- Scalability
- Load balancing

Map-Reduce and Hadoop

Word Count example



Map-Reduce

Inverted Index example

◆Input: (filename, text) records
◆Output: list of files containing each word

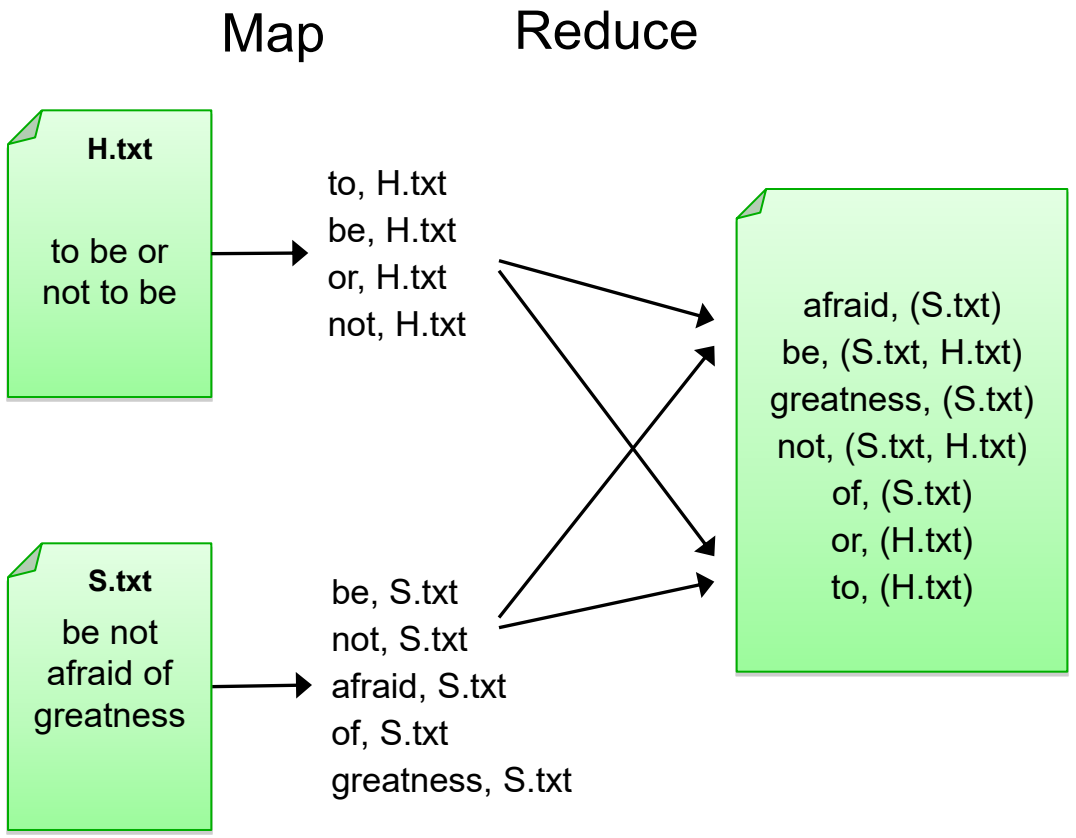
◆Map:

```
foreach word in text.split():
    output(word, filename)
```

◆Combine: uniquify filenames for each word

◆Reduce:

```
def reduce(word, filenames):
    output(word, sort(filenames))
```

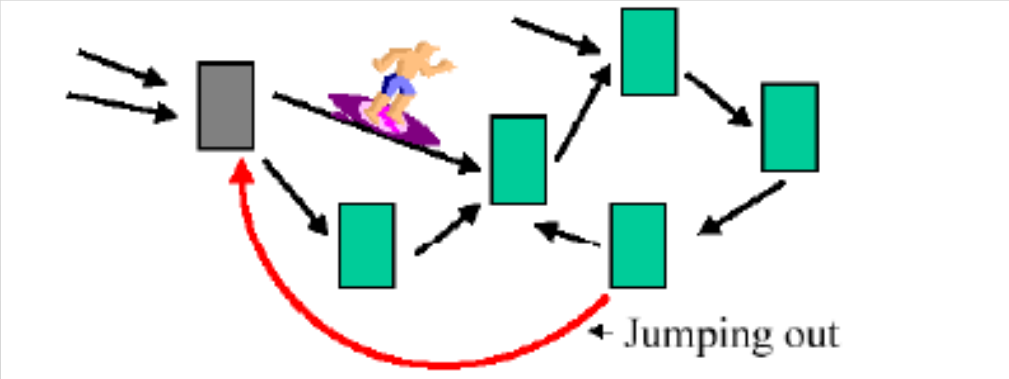


Google PageRank example

◆ Definition : Let D be the set of all Web pages. Let *I*(p) be the set of pages that link to the page p and let c_i be the total number of links going out of page p_i. The PageRank of page p_i, denoted by r_i, is then given by

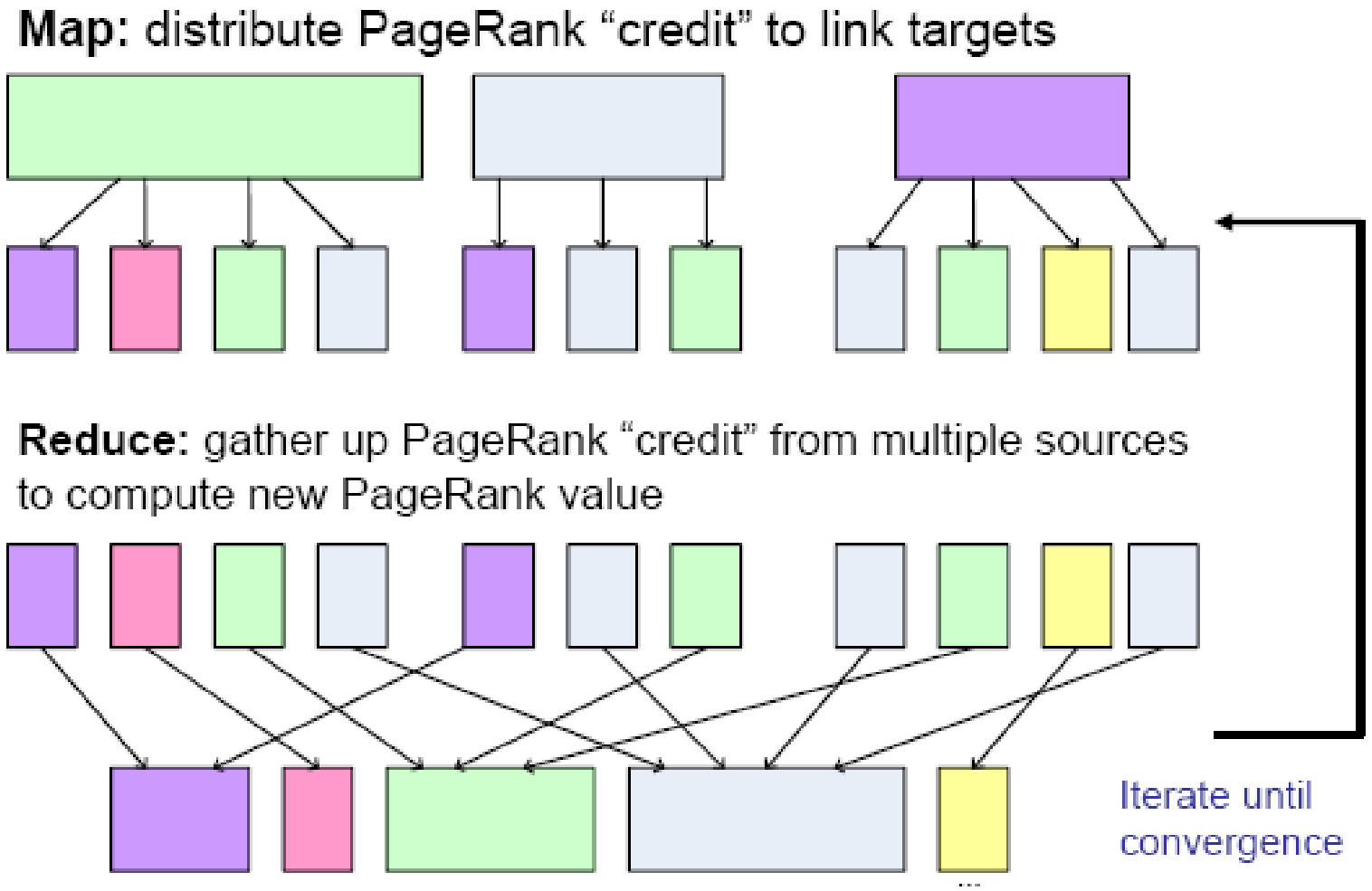
$$r_i = d \left[\sum_{p_j \in \mathcal{I}(p_i)} \frac{r_j}{c_j} \right] + (1-d)\frac{1}{|D|}$$

◆ Page Rank corresponds to *the probability distribution* of a random walk on the web graphs : *Random Surfer Model*



◆ Sometimes, a random surfer gets bored and jumps to a different page

Google PageRank example



Map-Reduce

Matrix example

Addition

- Input
  - Two matrices, A and B
- Map
  - Input: <[A|B], i, j, value>
  - Output: <key={i,j}, value=[A_{ij}|B_{ij}]>
- Reduce
  - Input: <key, a list of values={A_{ij}, B_{ij}}>
  - Output: <key={i,j}, value=A_{ij}+B_{ij}>

Multiplication

- Input
  - Two matrices, A (n x l) and B (l x m)
- Map
  - Input: <[A|B], i, j, value>
  - Output: <key=_____, value=_____>
- Reduce
  - Input: <key, a list of values>
  - Output: <key=_____, value=_____>

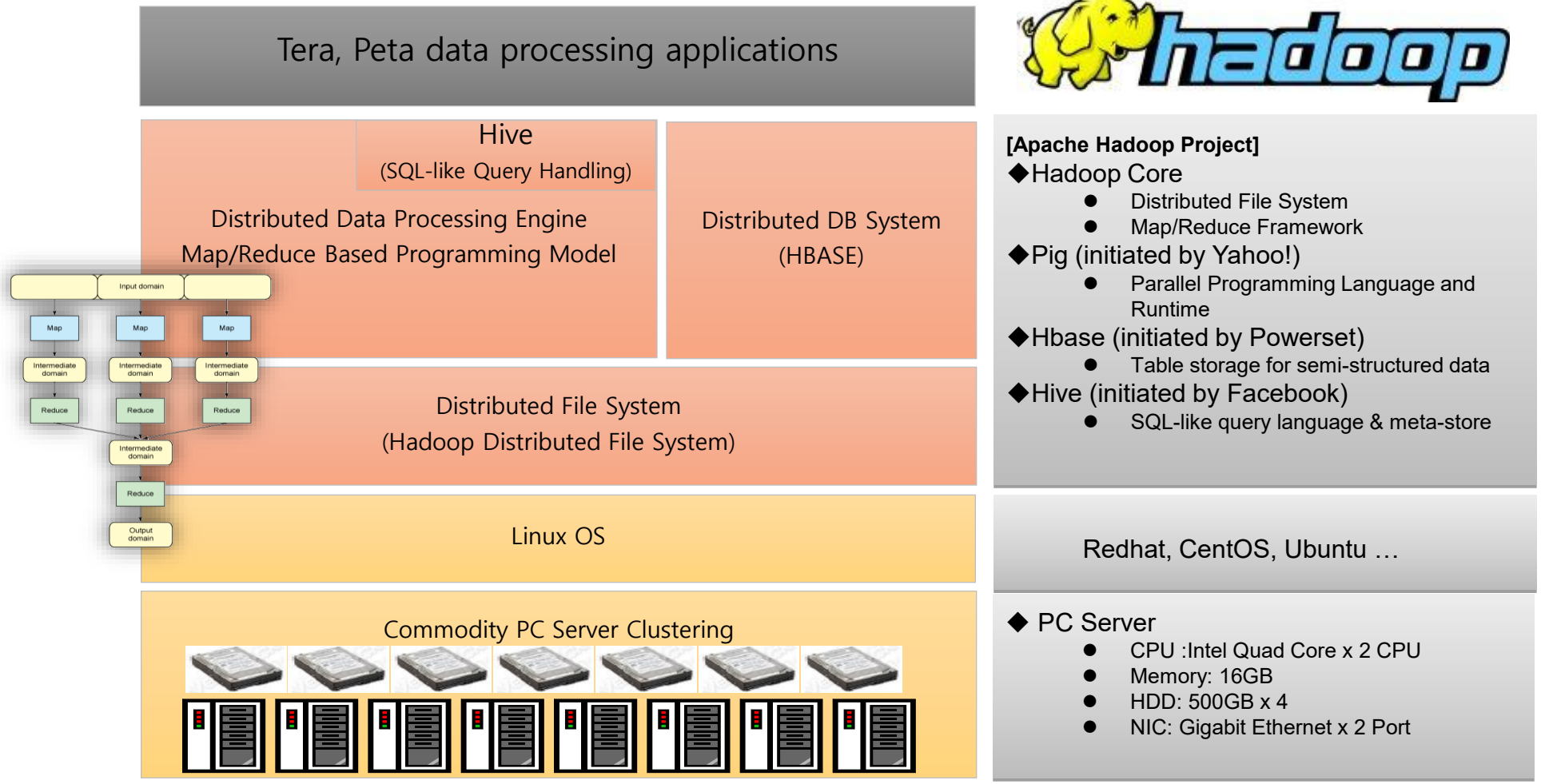
Contents

1. Cloud Computing
2. Map-Reduce
3. Hadoop
    - What is Hadoop?
    - HDFS
    - Principle of Hadoop's Map/Reduce Execution
    - Implementation of Word Count
4. Big Data Mining

Hadoop

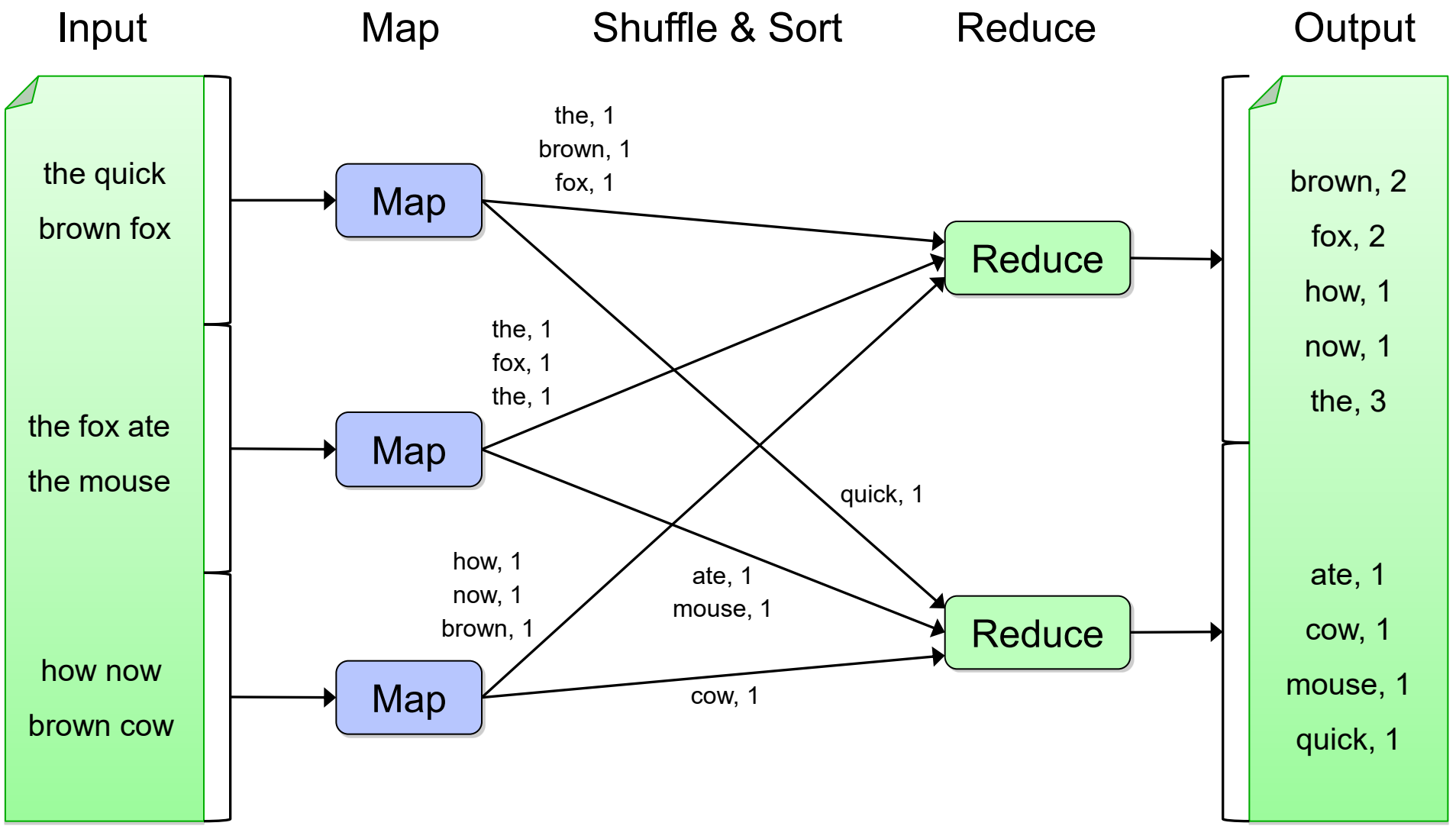
What is Hadoop?

Hadoop is a Open Source Platform based on Java. Hadoop has HDFS(hadoop Distributed File System) which is similar to GFS. Based on HDFS, Hadoop provides Map/Reduce Job execution on a cluster composed of thousands of PCs.



Map-Reduce

Word Count example



Implementation of Word Count

```
public class MapClass extends MapReduceBase
    implements Mapper<LongWritable, Text, Text, IntWritable> {

    private final static IntWritable ONE = new IntWritable(1);

    public void map(LongWritable key, Text value,
                    OutputCollector<Text, IntWritable> out,
                    Reporter reporter) throws IOException {
      String line = value.toString();
      StringTokenizer itr = new StringTokenizer(line);
      while (itr.hasMoreTokens()) {
        out.collect(new text(itr.nextToken()), ONE);
      }
    }
 }
```

Implementation of Word Count

```
public class ReduceClass extends MapReduceBase
    implements Reducer<Text, IntWritable, Text, IntWritable> {

    public void reduce(Text key, Iterator<IntWritable> values,
                       OutputCollector<Text, IntWritable> out,
                       Reporter reporter) throws IOException {
      int sum = 0;
      while (values.hasNext()) {
        sum += values.next().get();
      }
      out.collect(key, new IntWritable(sum));
    }
}
```

Implementation of Word Count

```
public static void main(String[] args) throws Exception {
    JobConf conf = new JobConf(WordCount.class);
    conf.setJobName("wordcount");

    conf.setMapperClass(MapClass.class);
    conf.setCombinerClass(ReduceClass.class);
    conf.setReducerClass(ReduceClass.class);

    FileInputFormat.setInputPaths(conf, args[0]);
    FileOutputFormat.setOutputPath(conf, new Path(args[1]));

    conf.setOutputKeyClass(Text.class); // out keys are words (strings)
    conf.setOutputValueClass(IntWritable.class); // values are counts

    JobClient.runJob(conf);
}
```